

UserFS

Benchea Matei & Mitoceanu Ciprian

Universitatea din București

Departamentul de informatică

Instrumente și Tehnici de Bază în Informatică

2025-2026

Outline

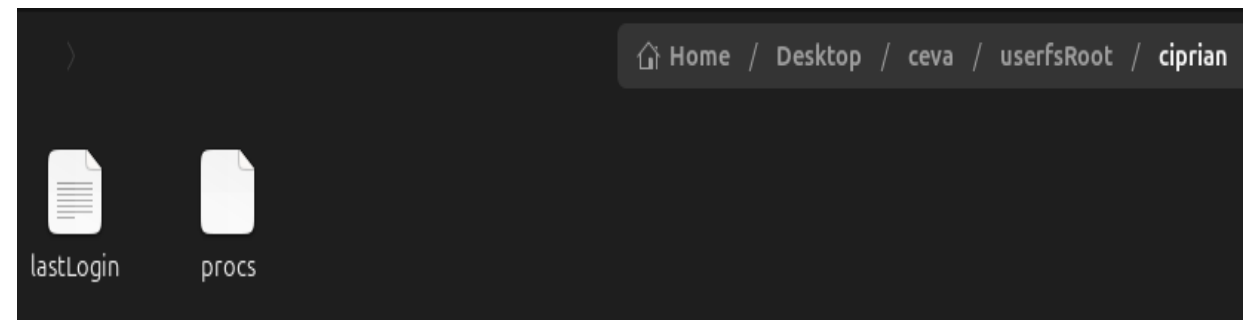
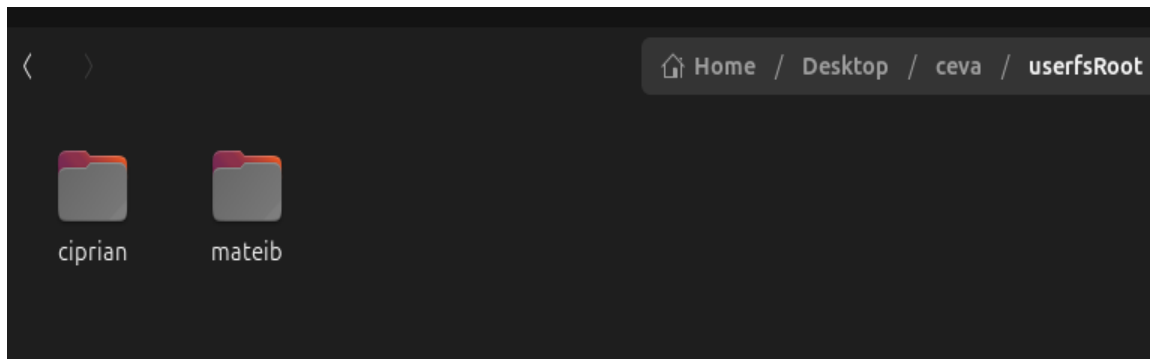
- Enunțul proiectului
- Descrierea problemei
- Particularitățile soluției
- Experimente
- Concluzii

Enunț

Scrieți un program (script) shell care reprezintă fiecare utilizator activ din sistem printr-un director în care se găsește un fișier ***procs*** care listează procesele curente ale utilizatorului. Toate aceste directoare care conțin informații despre un utilizator activ sunt grupate într-un același director, numit director rădăcină. Periodic, să zicem la fiecare 30s, informația despre utilizatorii activi se actualizează. Pentru utilizatorii care dispar din sistem în timp, directorul corespunzător va afișa un fișier ***procs*** gol și un fișier suplimentar ***lastlogin*** care afișează data ultimei sesiuni a utilizatorului din sistem.

Descrierea problemei UserFS (I)

- (Problema generală & datele particulare ale problemei) **Problema UserFS** constă în dezvoltarea unui script cu ajutorul căruia să monitorizăm utilizatorii unui sistem și activitatea acestora în mod dinamic și periodic. În particular, trebuie creat un director rădăcină (în cazul nostru el va purta numele de ***userfsRoot***) care să conțină câte un sub-director pentru fiecare utilizator care a fost logat pe sistem (sub-directorul va purta chiar numele utilizatorului) care să conțină la rândul lor câte un fișier ***procs*** unde să se găsească o listă cu toate procesele respectivului utilizator. În cazul în care un utilizator s-a delogat pe parcursul rulării scriptului, atunci în directorul său fișierul procs va fi gol, iar adițional va apărea un fișier ***lastLogin*** în care se va găsi data ultimei sesiuni a utilizatorului în sistem.



Descrierea problemei UserFS (II)

- (Obiective) Principalul obiectiv al UserFS este crearea unei unelte minimaliste pentru monitorizarea activității utilizatorilor logați pe un sistem. Mai mult de atât, vrem să putem ajuta utilizatorul UserFS prin punerea la dispoziție a unor operații de query pe datele culese de script (ca să nu fie nevoie de inspecția manuală a directoarelor sau a fișierelor).
- (Importanță) În mod fundamental, **UserFS** adresează problema **monitorizării activității** de pe un sistem. De ce este acest lucru important?
 - Ne poate ajuta să identificăm mai ușor activități suspicioase ale unor utilizatori;
 - Putem să identificăm ușor ce procese consumă excesiv date și respectiv ce utilizator le-a inițializat;
 - Putem să extragem mai ușor informații legate de activitatea unor utilizatori anume;
 - Ne ajută să realizăm statistici legate de activitatea de pe sistemul nostru.

Descrierea problemei UserFS (III) - Posibile soluții

- Soluții alternative (și încercate, vezi secțiunea de experimente):
 - Utilizarea *crontab* pentru programarea operației de update
 - Utilizarea *watch* pentru programarea operației de update
 - Utilizarea *while true; do ... + sleep 30*
- Soluția aleasă: scriptul nostru trimite operația periodică de update a informațiilor din directoare în background, permițând astfel implementarea unei interfețe interactive în terminal; prin intermediul căreia se apelează operațiile de query asupra datelor.

Particularitățile soluției (I) - Tehnologii cheie

- *who* – o comandă care ne furnizează o serie de date despre toți utilizatorii logați în sistem. Apelată fără flag-uri, ea afișează pe prima coloană numele utilizatorului, pe a doua, modul în care utilizatorul este logat pe sistem (tty/pts), pe următoarele două, data și ora logării, iar pe ultima adresa IP.
- *awk* – o unealtă puternică pentru manipularea și analiza fișierelor text (un limbaj de programare interpretat în sine).
- *sort* – o comandă care ajută la ordonarea liniilor dintr-un fișier. În particular o vom folosi împreună cu flag-ul *-u* care elimină duplicatele din rezultat.

Cele 3 de mai sus stau la baza funcționării operațiilor de update. În particular, (*who | awk '{print \$1}' | sort -u*) ne furnizează o listă cu toți utilizatorii logați în sistem la un moment dat.

- *ps* – o comandă care furnizează o listă cu informații legate de procesele curente din cadrul sistemului. În cazul nostru o vom folosi sub forma *ps -u nume*, care ne furnizează date doar despre procesele utilizatorului cu numele *nume*.
- Lansarea în background, *\$!* și *trap*.

Particularitățile soluției (II) - Design

- Soluția prezintă două mari secțiuni - Update & inițializare și Queries.
 - Update & inițializare - operația de update a datelor se realizează în background, ea terminându-se odată cu terminarea scriptului. Utilizatorul UserFS nu interacționează direct cu această parte a programului, el poate doar să observe urmările acesteia (fie prin intermediul operațiilor de query, fie prin verificarea efectivă a directoarelor create).
 - Queries - operațiile de queries sunt "cerute" direct din linia de comandă, acolo unde utilizatorul UserFS este întâmpinat de o interfață grafică care îi permite să apeleze una din cele 7 funcții de query oferite prin intermediul introducerii de la tastatură a unor comenzi (care sunt specificate în manualul programului).

```
man
-----UserFS-----
cnta - afiseaza numarul de utilizatori logati pe sistem
cnti - afiseaza numarul de utilizatori delogati de pe sistem
swa - afiseaza o lista cu utilizatorii logati pe sistem
swi - afiseaza o lista cu utilizatorii delogati de pe sistem
search <user> - afiseaza starea utilizatorului user (logat/delogat/nu a fost logat)
last <user> - afiseaza ultima data cand utilizatorul user s-a delogat de pe sistem
procs <user> [op] - afiseaza procesele utilizatorului user
    -a afiseaza toate procesele utilizatorului
    -n urmat de un numar x afiseaza ultimele x procese ale utilizatorului
clear - curata terminalul
exit - opreste UserFS
-----
swa
mateib
```


Particularitățile soluției (III) - Implementare (I)

- Inițializarea directorului rădăcină: verificăm dacă în sistem există deja un director cu numele `userfsRoot`. În caz afirmativ îl ștergem. Creăm apoi directorul `userfsRoot` – aici vom plasa toate directoarele utilizatorilor de pe sistem.
- Funcția de update: este formată în esență din două părți: funcția efectivă de update a datelor și așa-zisul wrapper pentru funcția de update. Funcția efectivă de update se ocupă cu modificarea conținutului fișierelor *procs* și eventual de crearea fișierelor *lastLogin*. Ea iterează prin lista utilizatorilor curenți (`who | awk '{print $1}' | sort -u`), iar apoi caută directorul respectiv în directorul rădăcină pentru a updata informația pentru utilizatorii logați. Ulterior iterează prin toate directoarele din `userfsRoot` și vede care dintre numele acestora nu se găsesc în lista de utilizatori logați, astfel actualizează datele pentru utilizatorii delogați. Funcția wrapper conține un loop infinit cu `while true`; în interiorul căruia se apelează funcția efectivă de update, iar apoi un `sleep 30` (pentru a actualiza datele odată la 30s). Funcția wrapper este trimisă în background, process id-ul acesteia fiind reținut. În momentul în care programul se termină, se face un kill pentru process id-ul funcției wrapper.

Particularitățile soluției (IV) - Cerințe minime de rulare

- Script-ul va rula pe un sistem Linux
- Necesități software: suport pentru Bash și awk
- Din puncte de vedere al HW nu există cerințe extra peste cele minime, programul nefiind unul care să necesite multe resurse pentru funcționare (pentru sisteme cu resurse hardware limitate recomandăm totuși să nu se folosească comanda `procs` cu flag-ul `-a`)

```
if [ -d "userfsRoot" ]; then
    rm -rf "userfsRoot"
fi
mkdir "userfsRoot"
```

Inițializarea directorului rădăcină

```
update() {
    while true; do
        update_data
        sleep 30
    done
}
```

```
update &
upd_pid=$!
trap "kill $upd_pid" EXIT
```

Funcția wrapper de update

```
update_data() {
    users=$(who | awk '{print $1}' | sort -u)
    for user in $users; do
        dirUser="userfsRoot/$user"
        if [ ! -d "$dirUser" ]; then
            mkdir -p "$dirUser"
        fi
        ps -u "$user" > "$dirUser/procs"
        if [ -f "$dirUser/lastLogin" ]; then
            rm "$dirUser/lastLogin"
        fi
    done

    for x in "userfsRoot"/*; do
        [ -e "$x" ] || continue
        curr=$(basename "$x")
        if ! echo "$users" | grep -q "^$curr$"; then
            >"$x/procs"
            date > "$x/lastLogin"
        fi
    done
}
```

Funcția efectivă de update

Particularitățile soluției (III) - Implementare (II)

- Interfața în terminal reprezintă în esență o buclă infinită care așteaptă mereu input de la tastatură. În funcție de ce introduce utilizatorul se va apela una dintre funcțiile de query sau se va preciza dacă inputul este invalid. Loop-ul se termină cu terminarea programului.

```
while true; do
    read -r ce pp fl nr
    case "$ce" in
        cnta)
            count_active_users
            ;;
        swa)
            show_active_users
            ;;
        cnti)
            count_loggedOut_users
            ;;
        swi)
            show_loggedOut_users
            ;;
        search)
            search_for_user "$pp"
            ;;
        last)
            last_seen_active "$pp"
            ;;
        procs)
            show_last_processes "$pp" "$fl" "$nr"
            ;;
        exit)
            break
    esac
done
```

Particularitățile soluției (III) - Implementare (III)

- Funcția `count_active_users` afișează pe ecran numărul de utilizatori logați pe sistem la acel moment. Ea numără câte directoare din `userfsRoot` nu conțin fișierul `lastLogin`.
- Funcția `show_active_users` afișează pe ecran numele utilizatorilor logați pe sistem la acel moment.
- Funcția `count_loggedOut_users` afișează pe ecran numărul de utilizatori delogați de pe sistem la acel moment. Ea caută utilizatorii care au un director creat în `userfsRoot` care conține fișierul `lastLogin`.
- Funcția `show_loggedOut_users` afișează pe ecran utilizatorii delogați de pe sistem la acel moment.
- Funcția `search_for_user` ne spune dacă utilizatorul cu un anumit nume (pe care îl introducem noi de la tastatură) este logat/delogat sau nu a intrat pe sistem. Ea caută numele utilizatorului în directoarele din `userfsRoot` și dacă îl găsește verifică dacă există fișierul `lastLogin`.
- Funcția `last_seen_active` afișează pe ecran conținutul fișierului `lastLogin` în caz că acesta există pentru un anumit utilizator (al cărui nume îl introducem de la tastatură).
- Funcția `show_last_processes` afișează, în funcție de flag-urile primite, ultimele procese ale unui utilizator specificat.

```
count_loggedOut_users() {  
    cnt=0  
    for x in "userfsRoot"/*; do  
        [ -e "$x" ] || continue  
        if [ -f "$x/lastLogin" ]; then  
            ((cnt++))  
        fi  
    done  
    echo "$cnt"  
}
```

```
count_active_users() {  
    cnt=0  
    for x in "userfsRoot"/*; do  
        [ -e "$x" ] || continue  
        if [ ! -f "$x/lastLogin" ]; then  
            ((cnt++))  
        fi  
    done  
    echo "$cnt"  
}
```

```
show_active_users() {  
    for x in "userfsRoot"/*; do  
        [ -e "$x" ] || continue  
        if [ ! -f "$x/lastLogin" ]; then  
            curr=$(basename "$x")  
            echo "$curr"  
        fi  
    done  
}
```

```

last_seen_active() {
    target=$1
    for x in "userfsRoot"/*; do
        [ -e "$x" ] || continue
        curr=$(basename "$x")
        if [[ "$curr" == "$1" ]]; then

            if [ -f "$x/lastLogin" ]; then
                cat "$x/lastLogin"
                return 0
            else
                echo "Utilizatorul $curr este activ"
                return 0
            fi
        break
    done
    echo "Utilizatorul $target nu a fost logat pe sistem"
}

```

```

show_loggedOut_users() {
    users=$(who | awk '{print $1}' | sort -u)
    for x in "userfsRoot"/*; do
        [ -e "$x" ] || continue
        curr=$(basename "$x")
        if ! echo "$users" | grep -q "^$curr$"; then
            echo "$curr"
        fi
    done
}

```

```

search_for_user() {
    target="$1"
    for x in "userfsRoot"/*; do
        [ -e "$x" ] || continue
        curr=$(basename "$x")
        if [[ "$curr" == "$1" ]]; then
            if [ -f "$x/lastLogin" ]; then
                echo "Utilizatorul $1 a fost logat pe sistem la un"
                return 0
            else
                echo "Utilizatorul $1 este logat pe sistem"
                return 0
            fi
        break
    fi
done
    echo "Utilizatorul $1 nu a fost logat pe sistem"
}

```

```

show_last_processes() {
    target=$1
    for x in "userfsRoot"/*; do
        [ -e "$x" ] || continue
        curr=$(basename "$x")
        if [[ "$curr" == "$1" ]]; then
            if [ -f "$x/lastLogin" ]; then
                echo "Utilizatorul $1 nu este logat pe sistem momentan"
                return 0
            else
                if [[ "$2" == "-a" ]]; then
                    cat "$x/procs"
                elif [[ "$2" == "-n" ]]; then
                    if [[ $3 =~ ^[0-9]+$ ]]; then
                        tail -n "$3" "$x/procs"
                    else
                        tail "$x/procs"
                    fi
                else
                    tail "$x/procs"
                fi
            fi
        fi
    done
    echo "Utilizatorul $1 nu a fost logat pe sistem"
}

```

Experimente (I)

- (De implementare) Programarea funcției efective de update cu ajutorul *crontab* – problema care reiese este că nu putem să avem un interval de sub 1 minut.
- (De implementare) Programarea funcției efective de update cu ajutorul *watch* – problemele care reies sunt acelea că funcția trebuie exportată, iar *watch* are o proprie interfață grafică în terminal.
- (De implementare) Implementarea funcțiilor de query cu ajutorul listei `users=$(who | awk '{print $1}' | sort -u)` - problema care reiese este că sistemul de directoare își pierde relevanța, există un offset dintre timpul (curent) la care raportează funcția de query și timpul (care poate să fie în urmă cu până la 30s) la care se face update pe sistemul de directoare.
- (De implementare) Programarea funcției efective de update cu ajutorul unui *while true; ... sleep 30* (fără a trimite în background) – problema care reiese este aceea că în acest caz implementarea unei interfețe în terminal devine aproape imposibilă (atunci când facem un *sleep* terminalul se blochează în esență pentru cele 30s).

Experimente (II)

- (Pure testing) - am logat și delogat succesiv mai mulți utilizatori pe același sistem pentru a testa toate funcționalitățile funcțiilor de query (precum și dacă se construiește corect structura de directoare).
- (Pure testing) - la rebootarea/repornirea sistemului, UserFS se oprește echivalent cu exit normal.

Concluzii

- Eficiența Automatizării: Proiectul demonstrează cum un script Bash de dimensiuni reduse poate substitui monitorizarea manuală, oferind o structură de date organizată și ușor de parcurs.
- Prospecte viitoare: extinderea funcțiilor de query (permiterea unor interogari mai avansate), implementarea unor funcții pentru generarea de statistici în legătură cu activitatea utilizatorilor, extinderea sistemului pentru monitorizarea consumului de memorie pentru fiecare utilizator.
- Q.E.D.